

Exploitation of Out-of-Band Package-Scoped Efficient Performance Indicator Telemetry in TDX

Secured Embedded Architecture Laboratory (SEAL)
Indian Institute of Technology Kharagpur

April 20, 2026

Contents

1	OOB Efficient Performance Indicator Metric Side-channel in Intel TDX	2
1.1	Abstract	2
1.2	Introduction	2
1.3	Threat Model	3
1.4	Establishing Leakage with the EPI Telemetry	4
1.4.1	Reverse Engineering and Characterization of EPI	4
1.4.2	Synchronizing in-band execution and out-of-band EPI measurements	4
1.5	Real-World ML Use Cases Leveraging Out-Of-Band Performance Telemetry	5
1.5.1	Vision and Language Transformer Architecture Leakage	5
1.5.2	Prompt-Topic Leakage:	6

Chapter 1

OOB Efficient Performance Indicator Metric Side-channel in Intel TDX

1.1 Abstract

Intel Trust Domain Extensions (TDX) provides strict hardware-enforced isolation for virtual machines, termed Trust Domains (TDs). It operates under a stringent threat model where all external entities are untrusted, spanning both the (1) in-band execution path, including the hypervisor, host OS, and co-tenant VMs, and (2) out-of-band (OOB) platform management channels. By virtualizing a TD’s performance-monitoring state, Intel TDX isolates performance-monitoring counters (PMCs) from untrusted entities, thereby eliminating traditional PMC-based side-channel attacks.

Previous work has shown that despite the isolation of a victim TD’s performance-monitoring state, secret-dependent leakage remains observable when the hypervisor and the TD are co-scheduled on the same logical core. Although the hypervisor cannot directly access the TD’s counters, the TD’s demand on shared microarchitectural resources perturbs the hypervisor’s own PMC traces, yielding a workload-correlated contention channel. In this work, we expand the threat surface beyond the same-core co-residency. We present the first security analysis and exploitation of the Efficient Performance Indicator (EPI) metric, a previously unexplored package-scoped metric exposed through Intel’s OOB management interface. By exploiting this novel EPI channel, we successfully infer the model architectures of pre-trained ViTs, Language Transformers, and LLMs deployed within a TD, achieving up to 99.53% accuracy and thereby breaching workload confidentiality. We further demonstrate prompt semantic leakage, whereby an adversary infers the sensitive topic of a user-provided prompt to a TD-hosted LLM with 82.39% accuracy, thereby compromising user privacy. Ultimately, our findings demonstrate that both confidentiality and privacy of TDs can be breached using just the privilege boundaries permitted by the TDX threat model, exposing a critical gap in its isolation guarantees.

1.2 Introduction

The increasing deployment of privacy-sensitive workloads, spanning healthcare analytics, financial inference, and proprietary AI models, on shared cloud platforms has heightened the importance of hardware-enforced confidential computing. Traditional security mechanisms, such as encryption and access control, are often insufficient because they cannot fully protect data when a system is actively processing it. To address these challenges, several Trusted Execution Environments (TEEs) have been developed, each tailored to different use cases and platforms. TEEs provide isolated execution environments that shield sensitive code and data from the rest of the system, even in the presence of potentially compromised operating systems or hypervisors. For instance, *Intel Software Guard Extensions (SGX)*¹ offers enclave-based protection in general-purpose computing environments. *ARM*

¹<https://software.intel.com/en-us/sgx>

*TrustZone*², on the other hand, provides a hardware-enforced separation between secure and non-secure worlds, making it suitable for mobile devices and embedded systems. Furthermore, nowadays TEEs have also been extended to specialized environments, such as *AMD’s Secure Encrypted Virtualization (SEV)*³ to protect virtual machines (VMs) in cloud settings.

Although TEEs are designed to protect the confidentiality and integrity of secure enclaves, previous research has shown that these TEEs are vulnerable to various attacks [MHHW18, SG20, SYG⁺19, WCP⁺17]. Specifically, Intel SGX has been successfully targeted by various attack categories, including cache, branch prediction, and software vulnerabilities [GESM17, SWG⁺17, CCX⁺20, BMW⁺18, QWL⁺21]. To address the scalability and deployment limitations of enclave-oriented designs in multi-tenant cloud environments, Intel introduced Trust Domain Extensions (TDX)⁴, elevating isolation to the virtual machine granularity through hardware-enforced Trust Domains (TDs).

A defining characteristic of virtualization-based TEEs, such as Intel TDX, is the active virtualization of hardware state that prior enclave-based designs left exposed. TDX virtualizes a TD’s performance-monitoring unit, isolating PMC register files and associated control MSRs from the untrusted entities. As a stark contrast, the performance monitoring unit (PMU) is *not* virtualized in AMD SEV-SNP (on Zen 3 and 4). It has been shown in [GWSG25] that the design decision not to virtualize PMCs is detrimental to the security of workload secrets residing within SEV-SNP. In Intel TDX, on the other hand, the TD PMCs are not accessible by an untrusted adversary. As [GWSG25] also states, it is an accepted fact that such a design strategy is sufficient to secure workload secrets residing within the TEE against PMC-based side channels. AMD also decided to support TDX-like PMC virtualization starting with Zen 5⁵.

Previous work [MSM⁺25], has shown that PMC virtualization, while necessary, is insufficient to eliminate performance telemetry-based leakage from TDX-protected workloads. Although TDX prevents access to a victim TD’s architectural PMC state, it does not eliminate indirect observability when a Virtual Machine Manager (VMM) and a TD share a logical core.

In this work, we demonstrate that the leakage surface extends beyond same-core co-residency. Modern Intel server platforms incorporate out-of-band management interfaces, such as Baseboard Management Controllers (BMCs), that operate outside the TD protection boundary. Through the Package Configuration Space (PCS) accessible via the BMC, Intel Platform Monitoring Technology (PMT)⁶ exposes unfiltered platform telemetry, including the Efficient Performance Indicator (EPI). EPI is the only performance package-scoped metric quantifying productive CPU cycles across the entire socket. Despite its presence in Intel server platforms, EPI has received no prior security analysis. We present the first systematic characterization and demonstrate that it enables both workload confidentiality and user privacy breaches in TDX environments without requiring co-location. We present the first reverse-engineering and security analysis of Intel’s Efficient Performance Indicator (EPI), a previously unexplored package-scoped metric accessible via out-of-band management interfaces. Unlike the in-band PMC leakage, EPI requires no co-location assumptions and enables inference of Vision Transformer, language Transformer, and LLM architectures, as well as topic-inference attacks against TD-hosted LLM services, thereby breaching both workload confidentiality and user privacy.

1.3 Threat Model

We adopt the standard Intel TDX threat model [Int21], in which a confidential workload executes within a TD and is protected against the host VMM/hypervisor, the host OS, co-tenant VMs/TDs, and external platform management entities on a multi-tenant cloud platform. We assume that the victim TD executes a confidential workload on a TDX-enabled platform with both ATTRIBUTES.PERFMON and ATTRIBUTES.DEBUG disabled. The attacker operates through the OOB management interface and therefore does not require being co-located with the victim TD. We model the adversary as a platform administrator with authenticated access to the server’s Baseboard Management Controller. Through the platform-management interface, the BMC can access package-scoped telemetry, including

²<https://developer.arm.com/documentation/100690/0200/ARM-TrustZone-technology>

³<https://www.amd.com/en/developer/sev.html>

⁴<https://cdrdv2-public.intel.com/690419/TDX-Whitepaper-February2022.pdf>

⁵<https://www.amd.com/en/resources/product-security/bulletin/amd-sb-3013.html>

⁶<https://www.intel.com/content/www/us/en/content-details/710389/intel-platform-monitoring-technology-intel-pmt-technical-specification.html>

the Efficient Performance Indicator (EPI). In this model, the adversary consists of two components: ① an in-band privileged process $\mathcal{A}_1$, running on any core of the same socket, which issues queries to the TD-hosted inference service to trigger victim workload execution, and ② an out-of-band management component $\mathcal{A}_2$, under administrator control, and collect sampled package-scoped EPI metric.

1.4 Establishing Leakage with the EPI Telemetry

1.4.1 Reverse Engineering and Characterization of EPI

Modern Intel Xeon server platforms incorporate a Baseboard Management Controller that communicates with the in-band processor package via the Platform Environment Control Interface (PECI). PECI defines a master-slave protocol where the BMC acts as the *PECI originator* and the processor acts as the *PECI responder*. Each processor socket is assigned a fixed PECI client address (0x30 for the primary socket). PECI exposes a structured command suite through which the BMC queries and configures the processor state. On Intel Xeon Sapphire Rapids, the supported command suite includes `Ping()`, `GetDIB()`, `GetTemp()`, `RdPkgConfig()`, `RdIAMS()`, `WrIAMS()` etc. From our attack perspective, we critically leverage the `RdPkgConfig()` command, which provides BMC read access to the processor’s Package Configuration Space (PCS). `RdPkgConfig()` is parameterized by an 8-bit `index` identifying the PCS service category and a 16-bit `parameter`, returning an 8-bit completion code and a 32-bit data value. A valid request returns 0x40 and an invalid index-parameter combination returns 0x90.

Among the package-scoped telemetry exposed by the PCS⁷ spanning energy, thermal, and performance domains, we identify `PECI_MBX_INDEX_EPI`, the Efficient Performance Indicator (EPI) (a previously unexplored metric in the security literature), as a performance metric that yields exploitable secret-dependent leakage from a Trust Domain. EPI quantifies the aggregate number of cycles during which the processor is productively engaged, i.e., cycles in which the processor is actively executing work toward instruction retirement. To characterize EPI, we issue `RdPkgConfig` requests with `index` set to `PECI_MBX_INDEX_EPI` (0x06). While the `index` values are partially documented, the corresponding `parameter` values that yield valid responses for each index are undocumented. We therefore design a systematic fuzzer to discover the exact (`index`, `parameter`) combination that returns valid EPI readings. We fix the `index` to `PECI_MBX_INDEX_EPI` and exhaustively enumerate all 2^{16} possible values of the 16-bit `parameter` field (i.e., 0x0000 through 0xffff). We execute a heavy matrix multiplication operation on the in-band side, and we issue a `RdPkgConfig` request from the out-of-band side and record: (i) the PECI completion code, and (ii) the returned EPI value.

• **Observation:** We observe that *all* 65,536 parameter values return `PECI_DEV_CC_SUCCESS` (0x40) with non-zero PCS values. No query returns an error completion code or a zero-valued response. Moreover, the returned values form a *strictly monotonically increasing* sequence across the entire sweep. This reveals two properties of the EPI metric: (i) the `parameter` field is not interpreted by the hardware for the EPI index, every query reads the same underlying counter regardless of parameter value, and (ii) the successive queries return progressively higher values as additional productive processor cycles accumulate between consecutive PECI queries.

1.4.2 Synchronizing in-band execution and out-of-band EPI measurements

Next, we describe how the EPI metric can be exploited as a side-channel vector for extracting secret-dependent information from a victim Trust Domain (TD). The above experiment was intended to characterize the properties of the EPI accumulator and, accordingly, relied on controlled workloads with precisely known execution boundaries. In a real attack setting, however, the adversary has no control over the code executing inside the victim TD. Consequently, the adversary must automatically identify the start and end of the victim workload in order to tightly bound the EPI measurement window around the target inference. To this end, we adopt the unidirectional signaling channel from the in-band VMM adversary, $\mathcal{A}_1$, to the out-of-band BMC adversary, $\mathcal{A}_2$, introduced in [MAB⁺25]. We first observe that this coordination is a natural consequence of TDX’s own threat model, *not an additional adversarial assumption*. Under TDX, the VMM is explicitly untrusted and lies outside the TD’s TCB. The VMM architecturally controls TD lifecycle management: it initiates TD entry

⁷<https://github.com/openbmc/linux/blob/dev-5.4/include/uapi/linux/peci-iocctl.h>

(via `SEAMCALL[TDH.VP.ENTER]`), regains control on TD exit, and observes all associated I/O activity. Likewise, the BMC operates on a physically separate management interface entirely outside a TD’s TCB. In commercial cloud deployments, the cloud administrator operates both the hypervisor and the BMC interface. The adversary ($\mathcal{A}_1, \mathcal{A}_2$), is therefore the cloud provider itself, precisely the TDX, that is designed to defend against. The signaling mechanism, repurposes a deliberately disabled performance monitoring counter (`IA32.PMC0`) as a signaling register between $\mathcal{A}_1$ and $\mathcal{A}_2$.

Methodology: Consider $\mathcal{V}$ is the victim TD, and the combined adversary is ($\mathcal{A}_1, \mathcal{A}_2$).

1. $\mathcal{A}_1$ disables `IA32.PMC0` by programming its control register `IA32.PERFVTSEL0` such that `IA32.PMC0` counts no architectural events. $\mathcal{A}_1$ ensures `IA32.PMC0` remains disabled throughout the duration of the attack. Meanwhile, $\mathcal{A}_2$ continuously polls the least-significant bit (LSB) of `IA32.PMC0` through `RdIAMSRR`.
2. $\mathcal{A}_1$ sets the LSB of `IA32.PMC0`. $\mathcal{A}_2$ detects this transition and begins *continuously* sampling the EPI metric via `RdPkgConfig` with `index = 6` and `parameter = 0x0000` at a fixed interval of 40 ms.
3. $\mathcal{A}_1$ initiates the execution of $\mathcal{V}$ on a designated CPU core. Additionally, pins high-priority threads with negligible workload intensity to all other cores in the package to suppress non-victim contributions to the package-scoped EPI accumulator.
4. While $\mathcal{V}$ executes, $\mathcal{A}_2$ continuously samples the EPI accumulator via `RdPkgConfig` at 40 ms intervals and computes the per-interval difference δ_i on-the-fly (Listing 1.1), producing a differential EPI trace $\langle \delta_1, \delta_2, \dots, \delta_{k-1} \rangle$, where k depends on the duration of $\mathcal{V}$ ’s execution.
5. Upon completion, control returns to $\mathcal{A}_1$, which clears the LSB of `IA32.PMC0` to 0. $\mathcal{A}_2$ detects this transition and terminates sampling.

Next, we use this setup to extract the architectures of ViT, Language Transformer, and LLM models being inferred within the TD, and to infer the sensitive topic of a user-provided prompt, ultimately compromising the data confidentiality of machine learning workloads despite TDX’s protection.

Listing 1.1: EPI Sampling by $\mathcal{A}_2$

```
RES1 <- peci_cmds RdPkgConfig 6 0x0000
while A1 has not signaled inference completion, do
  sleep(0.04)
  RES2 <- peci_cmds RdPkgConfig 6 0x0000
  delta_i <- RES2 - RES1
  RES1 <- RES2
end while
```

1.5 Real-World ML Use Cases Leveraging Out-Of-Band Performance Telemetry

1.5.1 Vision and Language Transformer Architecture Leakage

Previously, we demonstrated that in-band PMC traces are sufficient to fingerprint CNN architectures deployed within a victim TD. We now show that model architecture fingerprinting extends to the OOB setting, where **the adversary has no core co-location with the victim** and relies solely on the package-scoped EPI channel. Moreover, we target a substantially broader and more commercially relevant class of models: Vision Transformers (ViTs), language Transformers, and Large Language Models (LLMs) architectures that underpin the majority of modern MLaaS deployments. We consider the scenario in which a cloud tenant has deployed a pre-trained Transformer-based model within a TD, accessible only through an inference API. The model architecture is proprietary and constitutes the tenant’s intellectual property. The adversary ($\mathcal{A}_1, \mathcal{A}_2$), operating under Attacker Model 3 (Section ??), aims to identify which model is executing inside the TD using only the EPI traces captured during inference.

Table 1.1: Overall classification performance of the MLP classifier for ViT, Language Transformer and LLM architecture leakage under attacker model 3.

Family	Acc. (%)	F1-score	AUC
Vision Transformer Models (ViTs)	92.63	0.9235	0.9999
Language Transformer Models	99.31	0.9931	0.9999
Large Language Models (LLMs)	99.53	0.9953	1.0000

➤ **Target models:** We evaluate our attack against: (a) nine pre-trained ViT architectures spanning four vendors: Apple (MobileViT), Facebook (DeiT-Tiny, DeiT-Small, DeiT-Base), Google (ViT-Base, ViT-Large), and Microsoft (BEiT-Tiny, BEiT-Base, BEiT-Large). These models vary in embedding dimension, number of attention heads, and depth, producing diverse computational profiles during inference, (b) six multilingual Transformer architectures: MarianMT, T5-Small, T5-Base, T5-Large, XLM-RoBERTa-Base, and XLM-RoBERTa-Large. These models represent a range of encoder-decoders with varying parameter counts and attention configurations, and (c) six autoregressive LLMs: DistilGPT-2, OPT, Phi-3 Mini, Qwen, RWKV, and TinyLlama.

➤ **Methodology:** We operate under threat model as said above (Section ??), where the adversary ($\mathcal{A}_1$, $\mathcal{A}_2$) is the cloud administrator. The pre-trained target models are deployed within a TD and exposed to authorized users via an inference API. The measurement protocol follows Section 1.4.2, with the following adaptation in step 3, $\mathcal{A}_1$ triggers the victim inference by issuing an API query to the TD-hosted model on a designated CPU core. This is consistent with the MLaaS deployment model, where the model is exposed as a black-box inference service accessible to any authorized client. The adversary does not require any privileged access to the TD’s internals. It interacts with the model through the same API calls available. This is a standard assumption in the model stealing and fingerprinting literature [TZJ⁺16]. $\mathcal{A}_1$ additionally pins high-priority threads with negligible workload intensity to all other cores in the package to suppress non-victim contributions to the package-scoped EPI accumulator. $\mathcal{A}_2$ continuously samples the EPI accumulator via `RdPkgConfig` with `index = 6` and `parameter = 0x0000` at 40 ms intervals throughout the inference. At each sampling point, $\mathcal{A}_2$ reads the current EPI value and computes its difference from the previous reading on-the-fly, yielding the per-interval productive cycle count δ_i . Listing 1.1 presents the sampling procedure. Upon $\mathcal{A}_1$ signaling inference completion, $\mathcal{A}_2$ terminates sampling, producing a differential EPI trace $\langle \delta_1, \delta_2, \dots, \delta_{k-1} \rangle$ for the inference, where k depends on the inference duration.

➤ **Result and Analysis:** For each family, we perform 20 independent inferences per target model through API calls, collecting EPI traces per inference following the measurement protocol in Listing 1.1. Each trace is encoded as a string representation and constitutes a single instance in the dataset. We then follow the same three-step classification procedure as previous: (a) *Dataset creation:* For each model family, we build a separate dataset where each instance comprises the encoded EPI trace as the feature and the model architecture as the class label. This yields 180 instances for ViTs, 120 for language Transformers, and 120 for LLMs. Each dataset is split 80/20 into training and test sets with balanced class representation. (b) *Model building:* For each family, we train a separate MLP classifier consisting of one hidden fully-connected layer with 100 neurons and an output layer, using the Adam optimizer and ReLU activation. (c) *Evaluation:* We evaluate each classifier on its respective held-out test set. Table 1.1 reports the per-family results. The classifier achieves near-perfect accuracy on LLMs (99.53%) and language Transformers (99.31%), while ViTs yield a comparatively lower but still high accuracy of 92.63%. We attribute this difference to the relative separation in EPI magnitudes across architectures within each family.

1.5.2 Prompt-Topic Leakage:

The model fingerprinting attack targets *workload confidentiality* identifying *which* model is deployed inside the TD. We now demonstrate a strictly more fine-grained attack targeting *user privacy*, inferring the *sensitive topic* of a user-provided prompt to an LLM deployed inside a TD. Previous work [MO25] has demonstrated that prompt topics can be inferred from encrypted network traffic patterns during streaming LLM responses, achieving near-perfect classification across commercial LLMs. We demonstrate that an analogous leakage exists at the *package-scoped* interface, the EPI traces captured through the OOB management interface during TD-hosted LLM inference carries sufficient topic-dependent sig-

nal to distinguish sensitive prompts from noisy queries, even though the prompt content is encrypted in memory, and the adversary has no access to the TD’s internal state. Specifically, both the *trace length* k (the number of traces observed during the inference) and the *traces* vary as a function of the prompt topic: prompts on different topics elicit responses of different lengths and with different token-generation dynamics, producing distinguishable patterns in the rate and duration of productive cycle accumulation at the package level.

➤ **Target setup:** The victim tenant deploys Phi-3 Mini (3.8B parameters) inside a TD, exposed to authorized users through an inference API. We consider *breast cancer* as the sensitive target topic and generate 100 semantically diverse prompt variants on that topic. To represent non-target background traffic, we sample 5000 diverse, unrelated prompts, covering a broad range of topics. We introduce minor perturbations (random space insertions) into each query instance to prevent response caching across repeated queries of the same prompt.

➤ **Methodology:** The attack proceeds in two phases:

■ **Profiling Phase:** The adversary constructs a binary classifier that maps EPI traces to prompt topics. Since the TD-hosted model is exposed through a public inference API, $\mathcal{A}_1$ queries the *actual victim TD* with known prompts, just as any authorized user would. During each query, $\mathcal{A}_2$ simultaneously collects EPI traces following Listing 1.1. $\mathcal{A}_1$ selects 80 of the 100 target-topic prompts and queries each one of them 50 times, yielding 4,000 target traces, and it queries all 5,000 background prompts once each, yielding 4,000 background traces. Each trace is represented as a two-component feature: (i) the trace length k , and (ii) the trace content. Then we train a Bidirectional LSTM (Bi-LSTM) classifier with an attention mechanism on the 8,000 collected traces (4,000 target, 4,000 background), using Adam (lr=0.0002, batch size=32) with early stopping (patience=20).

■ **Attack Phase:** Once the classifier is trained, the adversary switches to passive observation. A legitimate user issues a prompt to the TD-hosted Phi-3 model through the inference API. The VMM ($\mathcal{A}_1$) observes the resulting TD entry and signals $\mathcal{A}_2$ via the disabled IA32.PMC0 register. $\mathcal{A}_2$ continuously samples EPI throughout the inference (Listing 1.1). Upon TD exit, $\mathcal{A}_1$ signals termination. $\mathcal{A}_2$ feeds the resulting EPI trace into the pre-trained classifier to determine whether the user’s prompt belongs to the sensitive target topic. At no point during this phase does the adversary issue or control the user’s prompt. This has direct security implications. An adversary can infer whether a TD-protected interaction concerns a sensitive topic without observing the prompt or protected TD memory, breaching the user’s privacy.

➤ **Result and Analysis:** To evaluate the classifier’s effectiveness in a realistic attack scenario, we simulate Phase 2 using the held-out prompts never seen during profiling: 20 target-topic phrasings (each queried 50 times, yielding 1,000 target traces) and 1,000 background prompts (queried once each). The Bi-LSTM classifier achieves **82.39%** accuracy with an F1-score of **0.8192** and an AUC of **0.9682** in distinguishing sensitive-topic prompts from background queries on the held-out test set. This implies that prompt semantics induce measurable, exploitable variations in the package-scoped EPI signal extending the leakage surface beyond architecture-level fingerprinting to reveal *input-dependent* behavior of the same model. An adversary with access to the OOB channel can therefore infer not only *which* model a tenant has deployed inside a TD, but also *what* the user is asking it about. This constitutes a direct breach of user privacy, despite TDX protection, an adversary can infer semantically meaningful attributes of a user’s query through an out-of-band performance telemetry.

Bibliography

- [BMW⁺18] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the intel SGX kingdom with transient out-of-order execution. In William Enck and Adrienne Porter Felt, editors, *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, pages 991–1008. USENIX Association, 2018.
- [CCX⁺20] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten-Hwang Lai. Sgxpectre: Stealing intel secrets from SGX enclaves via speculative execution. *IEEE Secur. Priv.*, 18(3):28–37, 2020.
- [GESM17] Johannes Götzfried, Moritz Eckert, Sebastian Schinzel, and Tilo Müller. Cache attacks on intel SGX. In Cristiano Giuffrida and Angelos Stavrou, editors, *Proceedings of the 10th European Workshop on Systems Security, EUROSEC 2017, Belgrade, Serbia, April 23, 2017*, pages 2:1–2:6. ACM, 2017.
- [GWSG25] Stefan Gast, Hannes Weissteiner, Robin Leander Schröder, and Daniel Gruss. Counter-seveillance: Performance-counter attacks on amd sev-snp. In *Network and Distributed System Security Symposium 2025: NDSS 2025*, 2025.
- [Int21] Intel Corporation. Intel® trusted domain extensions (intel® tdx). <https://cdrdv2-public.intel.com/690419/TDX-Whitepaper-February2022.pdf>, 2021.
- [MAB⁺25] Nimish Mishra, Kislay Arya, Sarani Bhattacharya, Paritosh Saxena, and Debdeep Mukhopadhyay. "oops!": Out-of-band remote power side-channel attacks on intel sgx and tdx. In *Proceedings of the 62nd Annual ACM/IEEE Design Automation Conference, DAC '25*. IEEE Press, 2025.
- [MHHW18] Mathias Morbitzer, Manuel Huber, Julian Horsch, and Sascha Wessel. Severed: Subverting amd’s virtual machine encryption. In *Proceedings of the 11th European Workshop on Systems Security*, pages 1–6, 2018.
- [MO25] Geoff McDonald and Jonathan Bar Or. Whisper leak: a side-channel attack on large language models, 2025.
- [MSM⁺25] Upasana Mandal, Shubhi Shukla, Nimish Mishra, Sarani Bhattacharya, Paritosh Saxena, and Debdeep Mukhopadhyay. Exploring side-channels in intel trust domain extensions. Cryptology ePrint Archive, Paper 2025/079, 2025.
- [QWL⁺21] Pengfei Qiu, Dongsheng Wang, Yongqiang Lyu, Ruidong Tian, Chunlu Wang, and Gang Qu. Voltjockey: A new dynamic voltage scaling-based fault injection attack on intel SGX. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 40(6):1130–1143, 2021.
- [SG20] Michael Schwarz and Daniel Gruss. How trusted execution environments fuel research on microarchitectural attacks. *IEEE Security & Privacy*, 18(5):18–27, 2020.
- [SWG⁺17] Michael Schwarz, Samuel Weiser, Daniel Gruss, Clémentine Maurice, and Stefan Mangard. Malware guard extension: Using SGX to conceal cache attacks. In Michalis Polychronakis and Michael Meier, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment - 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017*,

Proceedings, volume 10327 of *Lecture Notes in Computer Science*, pages 3–24. Springer, 2017.

- [SYG⁺19] Dimitrios Skarlatos, Mengjia Yan, Bhargava Gopireddy, Read Sprabery, Josep Torrellas, and Christopher W Fletcher. Microscope: Enabling microarchitectural replay attacks. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 318–331, 2019.
- [TZJ⁺16] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In *25th USENIX Security Symposium*, pages 601–618, 2016.
- [WCP⁺17] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2421–2434, 2017.